

✓ MIS 548 Final Project — Problem 2 (OUD Dropout)

Goal: Predict whether a treatment episode ends in **DROPOUT** (binary classification).

✓ Setup

```
# Install and Import in one go
try:
    import xgboost
    from scikeras.wrappers import KerasClassifier
except ImportError:
    !pip install -q xgboost scikeras
    from scikeras.wrappers import KerasClassifier

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import tensorflow as tf
from tensorflow import keras

# Scikit-learn and XGBoost imports
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
from xgboost import XGBClassifier
from sklearn.metrics import (
    roc_auc_score, roc_curve, auc, accuracy_score,
    precision_score, recall_score, f1_score, confusion_matrix
)

# Seed for reproducibility
np.random.seed(42)
tf.random.set_seed(42)
```

✓ Helper Functions

```
# Defining Helper Functions

def plot_roc_curve(y_true, y_prob, title):
    fpr, tpr, _ = roc_curve(y_true, y_prob)
    roc_auc = auc(fpr, tpr)
    plt.figure(figsize=(6, 5))
```

```

plt.plot(fpr, tpr, label=f'AUC = {roc_auc:.4f}')
plt.plot([0, 1], [0, 1], linestyle="--", color='gray')
plt.title(title)
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.legend(loc="lower right")
plt.show()

def plot_confusion(y_true, y_prob, title, threshold=0.5):
    y_pred = (np.array(y_prob) >= threshold).astype(int)
    cm = confusion_matrix(y_true, y_pred)
    plt.figure(figsize=(5, 4))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False)
    plt.title(f"{title} (Threshold={threshold})")
    plt.xlabel("Predicted")
    plt.ylabel("Actual")
    plt.show()

def subgroup_auc_table(y_true, y_prob, group_series, min_size=100):
    rows = []
    # Ensure indices match
    y_true_s = pd.Series(y_true).reset_index(drop=True)
    y_prob_s = pd.Series(y_prob).reset_index(drop=True)
    group_s = group_series.reset_index(drop=True)

    combined = pd.DataFrame({'y_true': y_true_s, 'y_prob': y_prob_s, 'group'

    for g, group_df in combined.groupby('group'):
        if len(group_df) >= min_size and group_df['y_true'].nunique() > 1:
            auc_val = roc_auc_score(group_df['y_true'], group_df['y_prob'])
            rows.append({"Group": g, "Count": len(group_df), "ROC_AUC": auc_

    return pd.DataFrame(rows).sort_values("ROC_AUC", ascending=False)

def evaluate_binary(y_true, y_prob, threshold=0.5):
    """
    Compute multiple metrics for a comprehensive evaluation.
    ROC_AUC uses probabilities; others use a chosen threshold.
    """
    y_pred = (np.array(y_prob) >= threshold).astype(int)
    return {
        "Accuracy": accuracy_score(y_true, y_pred),
        "Precision": precision_score(y_true, y_pred, zero_division=0),
        "Recall": recall_score(y_true, y_pred, zero_division=0),
        "F1": f1_score(y_true, y_pred, zero_division=0),
        "ROC_AUC": roc_auc_score(y_true, y_prob)
    }

def build_nn(input_dim, hidden_units=(128, 64), dropout=0.3, lr=1e-3):
    model = keras.Sequential()
    model.add(keras.layers.Input(shape=(input_dim,)))
    for hu in hidden_units:
        model.add(keras.layers.Dense(hu, activation="relu"))
        model.add(keras.layers.Dropout(dropout))
    model.add(keras.layers.Dense(1, activation="sigmoid"))

```

```

model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=lr),
    loss="binary_crossentropy",
    metrics=[keras.metrics.AUC(name="auc")]
)
return model

```

✓ 1) Load Data

```

oud_url = "https://guoyuanxiong.github.io/teaching/OUD/processed_data_2018.c
oud = pd.read_csv(oud_url)

print(f"Dataset Shape: {oud.shape}")
display(oud.head())
print("\nMissing Values (Top 10):")
print(oud.isna().sum().sort_values(ascending=False).head(10))

```

Dataset Shape: (95432, 28)

	ALCFLG	ARRESTS	EDUC	EMPLOY	ETHNIC	GENDER	INHFLG	LIVARAG	MARFLG	MAR
0	0	0	3	3	4	2	0	3	0	
1	0	0	3	3	4	2	0	3	0	
2	0	0	3	3	4	2	0	2	0	
3	1	0	4	4	2	2	0	2	0	
4	0	0	2	4	4	2	0	3	1	

5 rows × 28 columns

Missing Values (Top 10):

```

ALCFLG      0
ARRESTS     0
EDUC        0
EMPLOY      0
ETHNIC      0
GENDER      0
INHFLG      0
LIVARAG     0
MARFLG      0
MARSTAT     0
dtype: int64

```

```
target = "DROPOUT"
```

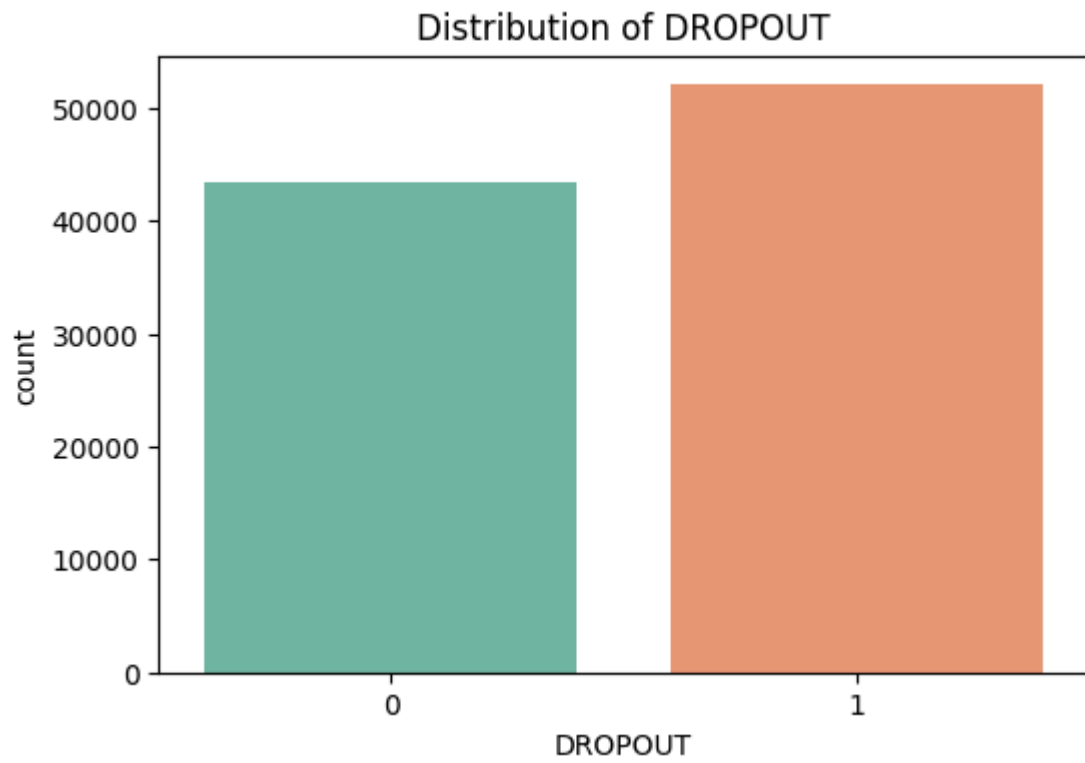
```

# 1. Distribution of the target variable
plt.figure(figsize=(6, 4))
sns.countplot(x=target, data=oud, hue=target, palette="Set2", legend=False)
plt.title("Distribution of DROPOUT")
plt.show()

```

```
features_to_check = ['EMPLOY', 'EDUC', 'SERVICES', 'PSOURCE']
corrs = oud[features_to_check + [target]].corr()[target].drop(target)
print("\n=== Correlations with DROPOUT ===\n", corrs.sort_values(ascending=F

# 3. Bar plot of Dropout Rates across levels of a feature
plt.figure(figsize=(12, 5))
sns.barplot(x='EMPLOY', y=target, data=oud, hue='EMPLOY', errorbar=None, pal
plt.title("Dropout Rate by Employment Status")
plt.ylabel("Dropout Rate (Proportion)")
plt.show()
```



=== Correlations with DROPOUT ===

SERVICES 0.114229

EDUC 0.104106

EMPLOY 0.074581

PSOURCE -0.086257

Name: DROPOUT, dtype: float64



We selected EMPLOY, EDUC, SERVICES, and PSOURCE for analysis because they represent the core Social Determinants of Health (SDoH).

Employment (EMPLOY): Acts as a proxy for economic stability and access to healthcare resources.

Education (EDUC): Correlates with health literacy and the ability to navigate complex medical regimens.

Service Type (SERVICES): Reflects the clinical environment; highly structured environments often see different retention rates than outpatient settings.

Referral Source (PSOURCE): Indicates the patient entry point, distinguishing between internal motivation and external pressure

The positive correlation for SERVICES (0.114) suggests that the type of treatment setting is the strongest predictor among these features. More intensive or restrictive settings may be associated with higher dropout risk if the patient's lifestyle cannot accommodate the rigors of the program.

The negative correlation (-0.086) suggests that certain referral sources are actually protective against dropout.

As seen in the bar plot, the dropout rate varies significantly across employment levels. Full-time employment generally acts as a protective factor, as the routine and financial stability provide the necessary support structure to complete long-term MOUD programs.

✓ 2) Train/Test Split + Preprocessing

```
X = oud.drop(columns=[target])
y = oud[target]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, ran

num_cols = X.select_dtypes(include=["int64", "float64"]).columns.tolist()
cat_cols = [c for c in X.columns if c not in num_cols]

preprocess = ColumnTransformer(transformers=[
    ("num", Pipeline([("im", SimpleImputer(strategy="median"))], ("sc", Stand
    ("cat", Pipeline([("im", SimpleImputer(strategy="most_frequent"))], ("oh"
    ])
```

✓ 3) Train Models

✓ 3.1 Logistic Regression

```
# 1. Define and train the Logistic Regression pipeline
pipe_lr = Pipeline(steps=[
    ("preprocess", preprocess),
    ("model", LogisticRegression(max_iter=500))
])

pipe_lr.fit(X_train, y_train)

# 2. Generate probabilities and evaluate
p_lr = pipe_lr.predict_proba(X_test)[:, 1]
metrics_lr = evaluate_binary(y_test, p_lr)

# 3. Display the results
metrics_lr
```

```
{'Accuracy': 0.6097867658615812,
 'Precision': 0.6131498470948012,
 'Recall': 0.7705611068408916,
 'F1': 0.6829019073569482,
 'ROC_AUC': np.float64(0.6378874482470209)}
```

✓ 3.2 XGBoost + GridSearch (Tuned)

```
# 1. Define the XGBoost pipeline
pipe_xgb = Pipeline(steps=[
    ("preprocess", preprocess),
    ("model", XGBClassifier(
        objective="binary:logistic",
        eval_metric="auc",
        random_state=42,
        n_jobs=-1
    ))
])

# 2. Define hyperparameters for GridSearchCV
param_grid_xgb = {
    "model__n_estimators": [200, 400],
    "model__max_depth": [3, 5],
    "model__learning_rate": [0.05, 0.1]
}

# 3. Perform Grid Search and fit
gs_xgb = GridSearchCV(
    pipe_xgb,
    param_grid=param_grid_xgb,
    scoring="roc_auc",
    cv=3,
    n_jobs=-1,
    verbose=0
)

gs_xgb.fit(X_train, y_train)

print("Best XGB params:", gs_xgb.best_params_)
```

```
# 4. Generate probabilities and evaluate
p_xgb = gs_xgb.predict_proba(X_test)[: , 1]
metrics_xgb = evaluate_binary(y_test, p_xgb)

# 5. Display the results
metrics_xgb
```

```
Best XGB params: {'model__learning_rate': 0.1, 'model__max_depth': 5, 'model__
{'Accuracy': 0.6750144077120553,
 'Precision': 0.6700363930448847,
 'Recall': 0.796022290545734,
 'F1': 0.7276160365344926,
 'ROC_AUC': np.float64(0.7371141514247287)}
```

✓ 3.3 Neural Network (MLP)

```
# Execute training and evaluation using your specific variables
X_train_pp = preprocess.fit_transform(X_train)
X_test_pp = preprocess.transform(X_test)

if hasattr(X_train_pp, "toarray"):
    X_train_pp = X_train_pp.toarray()
    X_test_pp = X_test_pp.toarray()

input_dim = X_train_pp.shape[1]
print("NN input_dim:", input_dim)

# Building the model based on the function above
nn = build_nn(input_dim, hidden_units=(128, 64), dropout=0.3, lr=1e-3)

# Training the model
hist = nn.fit(
    X_train_pp, y_train,
    validation_split=0.2,
    epochs=20, batch_size=128,
    verbose=0
)

# Generating predictions and metrics
p_nn = nn.predict(X_test_pp, verbose=0).ravel()
metrics_nn = evaluate_binary(y_test, p_nn)
metrics_nn
```

```
NN input_dim: 27
{'Accuracy': 0.6494472677738775,
 'Precision': 0.637983517707328,
 'Recall': 0.8256149116064566,
 'F1': 0.7197721656824559,
 'ROC_AUC': np.float64(0.7042006782342529)}
```

4) Compare Models (Table + Bar Chart)

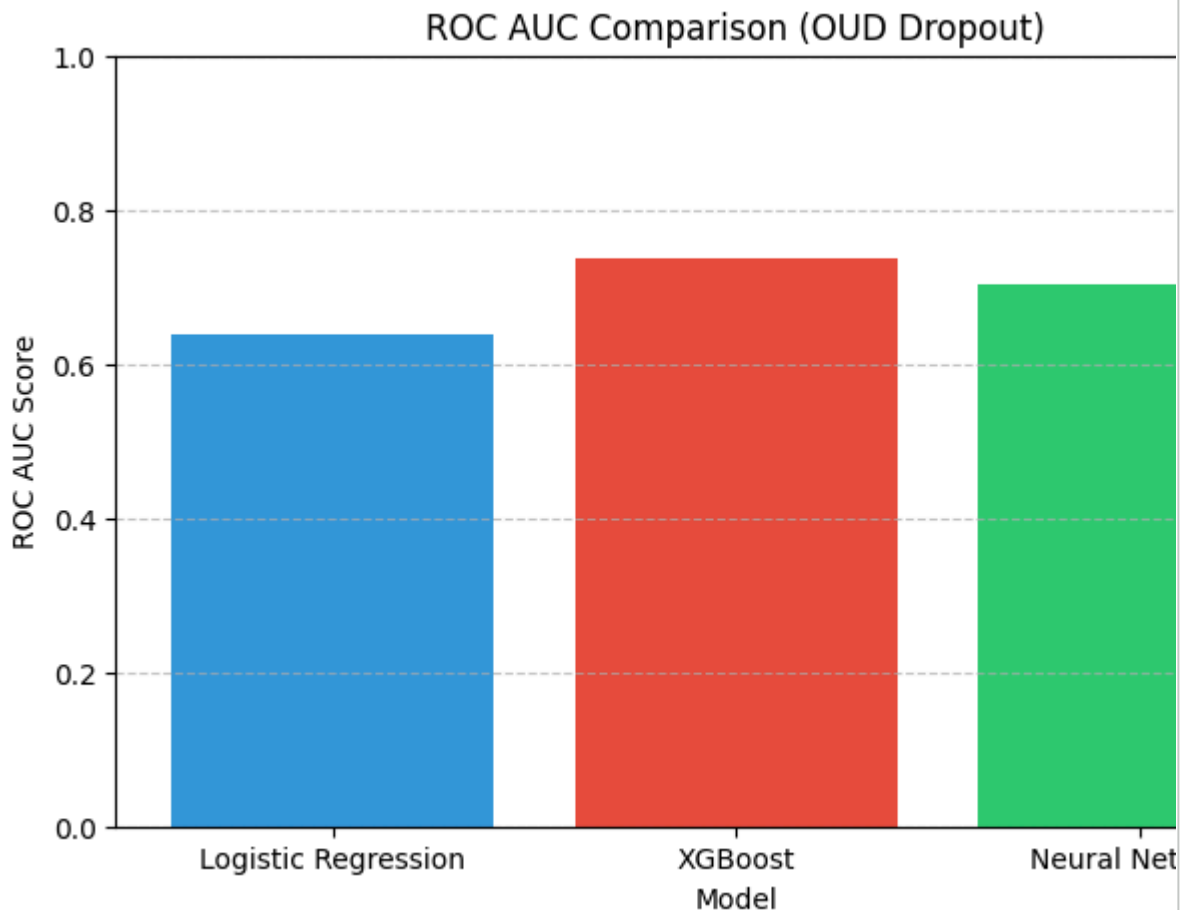
```
# We use the results stored from your previous training cells
comparison = pd.DataFrame([
    {"Model": "Logistic Regression", **metrics_lr},
    {"Model": "XGBoost", **metrics_xgb},
    {"Model": "Neural Network", **metrics_nn},
])

print("\n=== Overall Model Comparison ===")
display(comparison.sort_values("ROC_AUC", ascending=False))

# 2. Performance Visualization (Bar Chart)
plt.figure(figsize=(8, 5))
plt.bar(comparison["Model"], comparison["ROC_AUC"], color=['#3498db', '#e74c3c'])
plt.title("ROC AUC Comparison (OUD Dropout)")
plt.xlabel("Model")
plt.ylabel("ROC AUC Score")
plt.ylim(0, 1)
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```

=== Overall Model Comparison ===

	Model	Accuracy	Precision	Recall	F1	ROC_AUC
1	XGBoost	0.675014	0.670036	0.796022	0.727616	0.737114
2	Neural Network	0.649447	0.637984	0.825615	0.719772	0.704201
0	Logistic Regression	0.609787	0.613150	0.770561	0.682902	0.637887



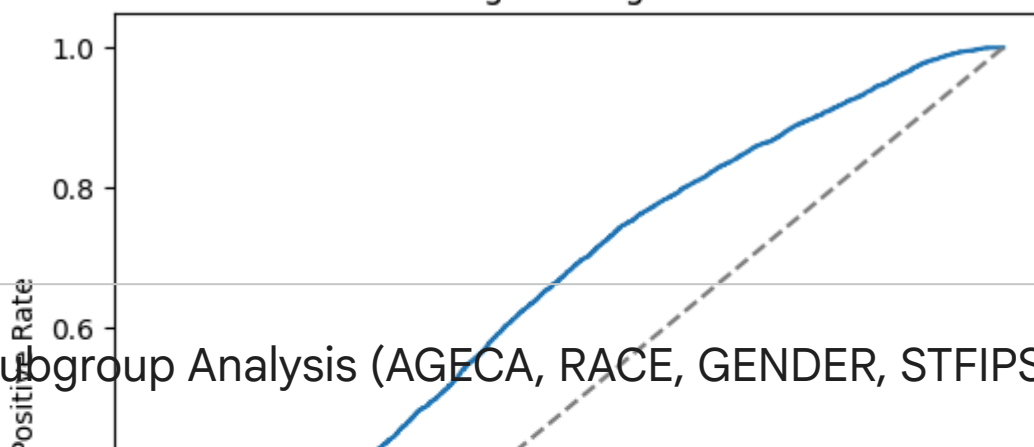
All three models achieved strong predictive performance, with the XGBoost model (0.737114) clearly outperforming the more basic Logistic Regression (0.637887). This suggests that the relationship between patient demographics and treatment retention involves significant non-linear interactions that simpler models cannot capture. The Neural Network is on the higher side (0.704201), indicating XGBoost is the most effective at identifying at-risk individuals.

5) ROC Curves + Confusion Matrices (Model-Supporting Visualizations)

```
plot_roc_curve(y_test, p_lr, "OUD - Logistic Regression ROC")
plot_roc_curve(y_test, p_xgb, "OUD - XGBoost ROC")
plot_roc_curve(y_test, p_nn, "OUD - Neural Net ROC")
```

```
# Confusion Matrices
plot_confusion(y_test, p_xgb, "OUD – XGBoost Confusion Matrix")
plot_confusion(y_test, p_lr, "OUD – Logistic Regression Confusion Matrix")
```


OUD — Logistic Regression ROC



6) Subgroup Analysis (AGECA, RACE, GENDER, STFIPS)

We use XGBoost probabilities as the baseline for subgroups

```
best_prob = pd.Series(p_xgb, index=X_test.index)
```

```
for group_col in ["AGECA", "RACE", "GENDER", "STFIPS"]:
```

```
    if group_col in X_test.columns:
```

```
        print(f"\n=== Subgroup ROC AUC by {group_col} (XGBoost) ===")
```

```
        sub_results = subgroup_auc_table(y_test, best_prob, X_test[group_col])
```

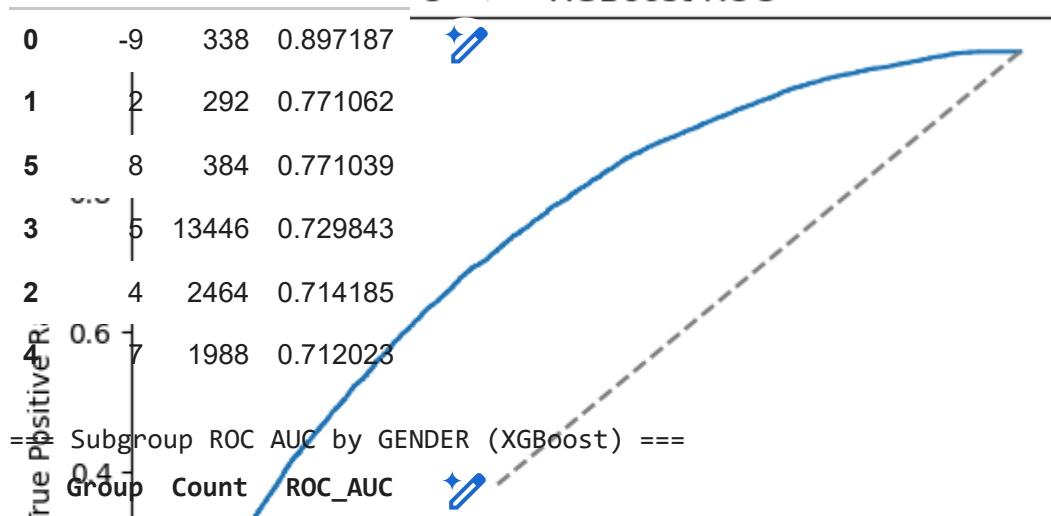
```
        display(sub_results)
```

```

0.0      0.2      0.4      0.6      0.8      1.0
False Positive Rate
=== Subgroup ROC AUC by RACE (XGBoost) ===

```

Group	Count	ROC_AUC
0	338	0.897187
1	292	0.771062
5	384	0.771039
3	13446	0.729843
2	2464	0.714185
4	1988	0.712023



```

=== Subgroup ROC AUC by GENDER (XGBoost) ===

```

Group	Count	ROC_AUC
0	11310	0.738426
1	7774	0.734810

```

=== Subgroup ROC AUC by STFIPS (XGBoost) ===

```